

SECURITY OPERATION CENTER PROJECT

Done by:

Omar Ayesh → 0226526

Ahmad Hasoneh → 2210256

Supervised by Dr. Oraib AbuAlganam



What is soc

A **Security Operations Center (SOC)** is a centralized unit that is responsible for continuously monitoring, detecting, analyzing, and responding to cybersecurity threats , attacks and potential risks within an organization. The SOC collects data from multiple sources such as endpoints (devices) , servers, network devices such as switches and routers , and security tools, then analyzes and correlate this data to identify and find suspicious or malicious activity and evidence.

Why organizations need monitoring

Organizations and companies today face a wide range of cyber threats , including phishing attacks (by emails or fake website), ransomware (encrypt files), insider threats (which is harder to detect), and lateral movement within internal networks . Many of these attacks are stealthy (Using Ai) and can remain undetected for long periods (persistence) if continuous monitoring and auditing is not in place.

Security monitoring is essential because :

- Attackers often bypass traditional security controls (need stronger authentication and security controls)
- A single compromised system can lead to wider network compromise (one vulnerable point can lead to catastrophic impacts)
- Early detection and responding reduces financial and operational damage
- Logs and alerts provide evidence for investigation and forensic examination.

Project Objectives

The main objective of this project is to design , develop and implement a SOC environment that demonstrates how cyberattacks can be detected and analyzed using security monitoring tools.

The specific objectives of this project are:

- To build a virtual SOC lab environment using multiple virtual machines([vmware](#)/[virtual box](#))
- To monitor a Windows OS endpoint device using [Sysmon](#)
- To collect and centralize logs using [Splunk](#) as a SIEM platform
- To simulate phishing-related risks and user awareness scenarios

- To demonstrate ransomware-like behavior
- To detect lateral movement attempts using a network honeypot
- Dealing with multiple sources of logs and correlate them together.
- To visualize security events using Splunk dashboards



Tools & Technologies :

Our project utilizes a combination of multiple monitoring and security tools, a security event management platform, network techniques, and web technologies to simulate and show how to detect cyberattacks and visualize them to know how to respond , all within a Security Operations Center (SOC) environment. Each tool was selected based on its relevance to real-world SOC operations.

1) Splunk (Security Information and Event Management – SIEM)

Splunk enterprise is used as the **central SIEM platform** in this project. It is responsible for collecting, storing , indexing, searching, and visualizing security logs generated from different sources (WIN machine , Honeypot) within the environment.

In this project, Splunk is hosted in **"KALI Linux"** performs the following functions:

- Give a central view of the logs for different endpoints
- Enables searching and analysis of security events and logs
- Correlates Logs , evets and network activity
- Helps in detected attacks through dashboards



2) Splunk Universal Forwarder

The **Splunk Universal Forwarder** is a lightweight agent installed on the

“ **Windows machine** “ , Its purpose is to collect & forward logs to the Splunk server for centralized analysis.

In this project, the Universal Forwarder is used to:

- Forward **Sysmon**-generated Windows Event Logs
- Send endpoint activity to the Splunk server in real time



3) Sysmon (System Monitor)

Sysmon is a Windows system service created by “ **Microsoft** “ that provides **detailed monitoring visibility on every endpoint activities (Application , CMD , security , system)** . It generates high-quality logs that are critical for detecting and responding to malicious behavior or any attack attempts .

Sysmon is used in this project to monitor:

- Process creation and execution
- Network connections or scanning
- File creation ,deletion or modification
- Attacker moves in the compromised machine



Sysmon allows SOC analysts to identify suspicious behavior such as ransomware activity and lateral movement attempts.

4) Honeypot (Pentbox)

A honeypot is a decoy system “ **Trap**” designed to attract(try to get the attacker there) and detect malicious activity using a non original(non-critical system) . In this project, **Pentbox** HP is used as a lightweight honeypot running on **Kali Linux**

The honeypot is configured to:

- Listen for incoming network connections
- Log access attempts from compromised systems

HONEYPOT
PENTBOX Lab



- Detect lateral movement attempts coming from compromised systems By monitoring interactions with the honeypot
- Therefor SOC analysts can identify unauthorized access attempts that indicate attacker movement beyond the initial compromise.

Lab Environment & Architecture

This project was implemented in a virtualized SOC lab environment using virtual box and Vmware designed to safely simulate cyberattacks and infected machines and analyze them without affecting real systems. The lab architecture separates two sides mainly the victim (Windows machine) endpoint and on the other hand Kali linux machine for splunk and honeypot while allowing controlled communication between them.

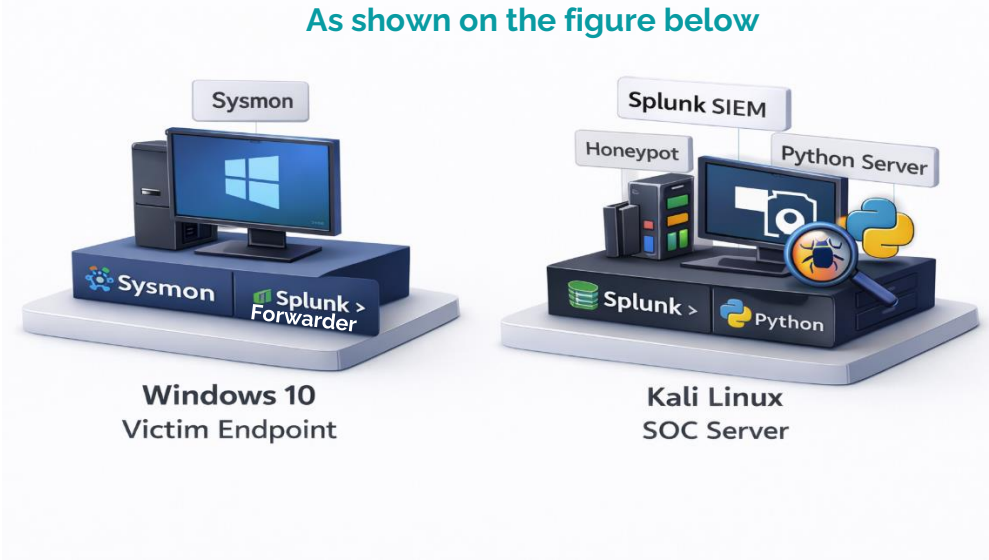
The environment consists of two main virtual machines connected within the same internal network to enable realistic attack simulation and monitoring.

Virtual Machine Overview

The lab environment includes the following virtual machines:

Virtual Machine	Operating System	Role
VM 1	Windows 10	Victim Endpoint (Sysmon + Splunk Universal Forwarder)
VM 2	Kali Linux	SOC Server (Splunk SIEM + Honeypot + Python Server)

As shown on the figure below



The **Windows virtual machine** represents a typical employee workstation and is intentionally monitored via Sysmon to capture and see endpoint activity . The **Kali Linux virtual machine** acts as the SOC server responsible for collecting, analyzing, and visualizing security logs found on that employee machine . Both virtual machines are connected to the **same network**, allowing them to communicate directly and in real time .

Components role:

Each security component is deployed and implemented to serve a specific role based on its operations :

- **Sysmon:** is installed on the Windows machine to generate detailed security logs related to process execution, network connections, and file activity (delete , create , modify).
- **Splunk Universal Forwarder** runs on the Windows machine to forward Sysmon logs to the SOC server so it can be seen in Splunk.
- **Splunk SIEM** runs on the Kali Linux machine and serves as the centralized log which collect from multiple sources for analysis and visualization(Dashboard).
- The **honeypot** is deployed on the Kali machine to detect unauthorized access attempts for example the lateral movement tactic .
- A **Python-based web server** is hosted on the Kali machine to simulate phishing-related scenarios and showing fake websites .

System Setup & Configuration

This section describes how the SOC lab environment was configured and installation of the tools to collect, forward, and analyze security logs. The setup focuses on establishing reliable , accurate and real time log collection from the endpoints and enabling centralized analysis using Splunk.

➤ Splunk Setup (SOC Server – Kali Linux)

Splunk Enterprise was installed on the Kali Linux machine to act as the **central SIEM solution platform**. After installation, the Splunk service was started from command line : **[sudo /opt/splunk/bin/splunk start --accept-license]** on the local host and accessed through its web interface .

Key configurations included:

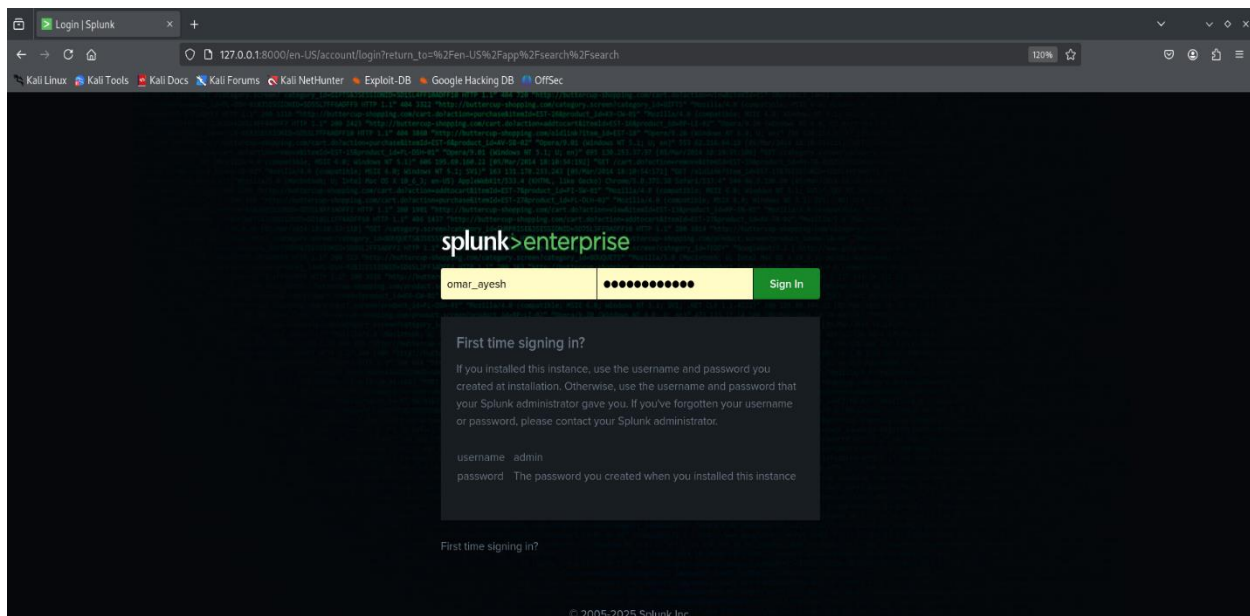
- Splunk Web Interface accessed via **port 8000 (the default port for splunk)**
- Log receiving enabled on **TCP port 9997** to accept data from forwarders so the universal forwarder in the windows machine will connect via port 9997

We can check in splunk is listening on port 9997 using this command :

Kali cmd : **[sudo lsof -iTCP:9997 -sTCP:LISTEN]**

- Use of the default **main index** for storing security events

Splunk was configured to listen for incoming connections from the Windows endpoint, allowing real-time ingestion of endpoint logs and then correlate them .



➤ Splunk Universal Forwarder Configuration (Windows machine)

The Splunk Universal Forwarder was installed on the Windows virtual machine to forward security logs and any malicious activities or attack behavior to the Splunk server.

The forwarder was configured to:

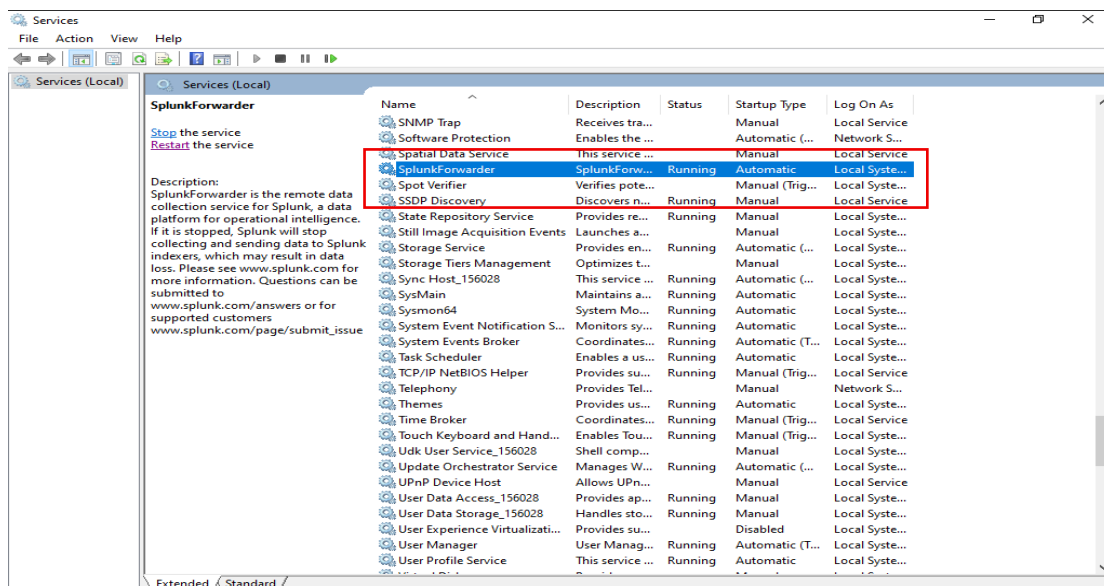
- Send data to the SOC server using **TCP port 9997**
- Use the Kali Linux machine's IP address as the receiving indexer
- Run as a background service to ensure continuous log forwarding and forward any activity to the splunk port in real time .
- To check the connection we use this command line :

[Test-NetConnection <KALI_IP> -Port 9997]

The forwarder (UF) configuration ensured that endpoint logs were reliably transmitted to Splunk without significant performance impact on the system because it is considered lightweight .

❖ **inputs.conf file** : The inputs.conf file is used to define what data sources the Universal Forwarder should monitor and collect in order to know what to forward . In this project, inputs.conf was configured to monitor Windows Event Logs, with particular focus on the Sysmon operational channel.

❖ **outputs.conf file** : The outputs.conf file is responsible for defining where the collected data is forwarded. So putting the **ip** of the kali machine .



➤ Sysmon Installation and Configuration

Sysmon was installed on the Windows endpoint from Microsoft to provide detailed, continuous and efficient visibility into system activity. Sysmon operates as a system service and records detailed system activity. During installation, a configuration file was applied to define which events should be monitored.

Sysmon Configuration (SwiftOnSecurity)

Sysmon was installed using a predefined configuration file based on the “**SwiftOnSecurity**” Sysmon configuration, which is widely used and configured in defensive security environments. This configuration is designed to balance between two concepts: visibility and noise reduction (so many unwanted logs will be overwhelming), ensuring that relevant security events and the ones will be needed in monitoring and investigation are captured without overwhelming analysts.

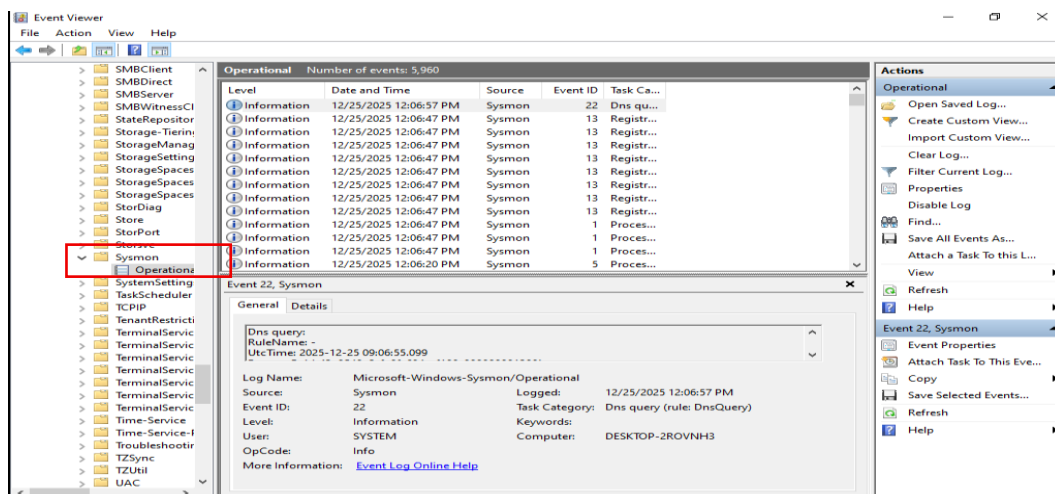
The **SwiftOnSecurity** configuration enables logging for:

- **Process creation and execution** (including their information and PID)
- **Network connections** (scanning or other Time wait, established status)
- **File** creation, deletion or modification.
- **Registry** modifications (critical changes can lead to huge compromise)
- **Persistence-related activities** (attackers try to remain there as possible)

Using a structured and well-prepared configuration ensures that Sysmon logs and events are consistent, meaningful, and suitable for SIEM analysis or correlation.

Key Sysmon Event Types:

- **Event ID 1 – Process Creation**
- **Event ID 3 – Network Connection**
- **Event ID 11 – File Creation**
- **Event ID 23 – File Deletion**
- **Event ID 4624 successfully logged on**
- **Event ID 4625 logon attempt failed**



Sysmon Event Log Channel

Sysmon logs are written to a dedicated Windows Event Log channel:

[Microsoft-Windows-Sysmon/Operational]

This separation from standard system logs (not at the same place) allows SOC analysts to focus on security-relevant alerts without interference from unrelated or not useful operating system events.

○ Honeypot Setup (Pentbox) Version 8.1

A lightweight honeypot was deployed and installed on the Kali Linux machine using Pentbox. The honeypot was configured to listen on a designated network port and record incoming connection attempts in separate chosen log file.

Key honeypot characteristics:

- Listened on a commonly targeted service port (e.g., HTTP) on **port 80**
- Logged source IP addresses and timestamps (to know who and when)
- Stored logs locally for ingestion into Splunk

We use these commands to setup the honeypot:

Run the HP: **[sudo ruby pentbox.rb]**

Network Tools → **Honeypot**

Manual configuration mode was selected

○ The honeypot was configured with the following parameters:

- **Listening Port:** 80
- **Service Type:** Network service honeypot
- **Response Message:** A generic denial message to avoid revealing system details "access denied"
- **Logging:** Enabled → to **/var/log/pentbox.log**

➤ Python HTTP Server Setup (Phishing Simulation)

A Python-based HTTP server was configured on the Kali Linux machine to simulate phishing-related scenarios to make a fake website. The server hosted a simple web page and downloadable content (the malware).

Key aspects of this setup:

- Server ran on a non-standard port to avoid conflicts
- Used for phishing simulation
- Allowed observation of user interaction and file downloads
- Implemented using this command line :

```
[ python3 -m http.server 8080 ]
```

This setup demonstrated how phishing can serve as an initial attack vector in real-world environments.

Detection & Monitoring Strategy “how the SOC sees things”

This project uses a layered SOC monitoring model.

Instead of relying on one security tool, multiple monitoring layers work at the same time to detect suspicious behavior.

Each layer answers a different security question, and together they provide full visibility.

“What is the computer doing?”

Endpoint Monitoring Layer (Sysmon)

This layer focuses on what is happening on the Windows machine. It monitors:

- Programs being executed
- Files being created or modified

This layer helps detect:

- Phishing execution
- Ransomware behavior
- Suspicious processes

“Where is the computer trying to go?”

Network & Deception Layer (Honeypot)

This layer focuses on how the compromised machine interacts with the network.

The honeypot:

- Looks like a real internal service
- Should never be accessed by normal users
- Logs any connection attempt

This layer helps detect:

- Lateral movement
- Internal reconnaissance
- Unauthorized access attempts

“Where does the SOC see everything?”

Centralized Monitoring Layer (Splunk SIEM)

This layer collects logs from:

- Sysmon (endpoint activity)
- Honeypot (network activity)

Splunk:

- Stores all logs in one place for better decision
- Allows searching and visualization
- Enables investigation

“How does the SOC confirm it's an attack?”

Correlation Layer (SOC Analysis)

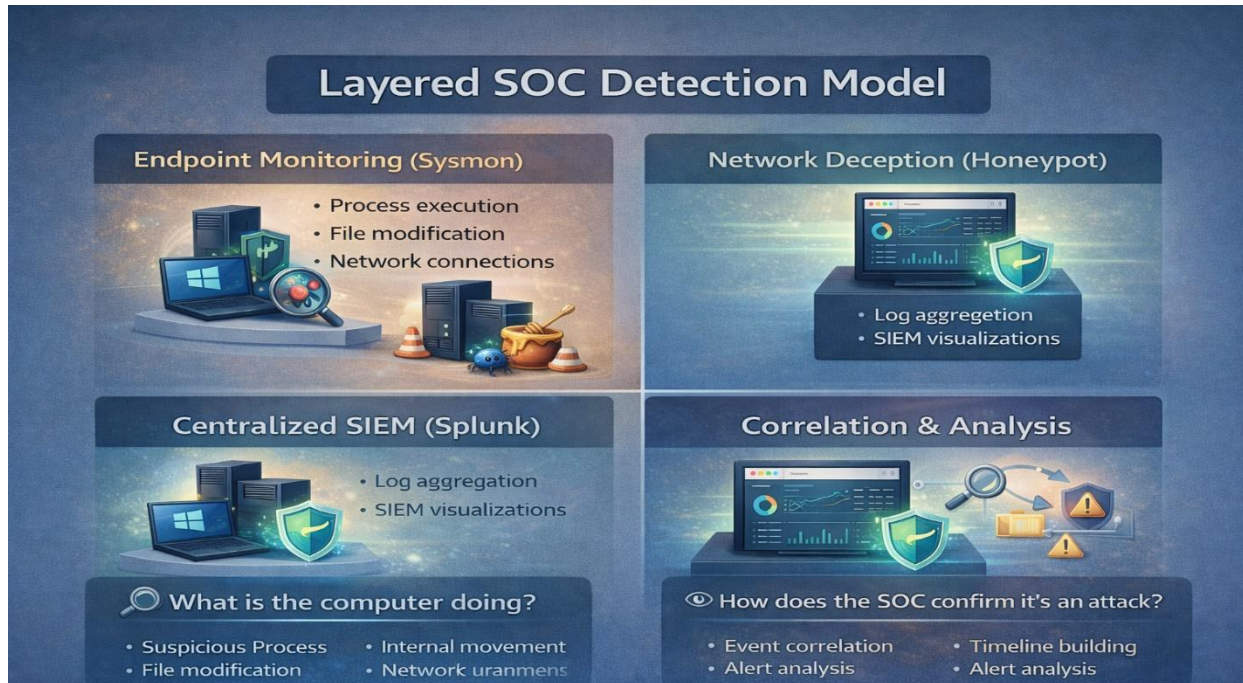
This layer connects events together.

Instead of looking at logs separately (one event may indicate nothing), the SOC:

- Matches timestamps to find a time flow of the attack
- Matches source IPs
- Builds attack context (try to know what the attacker did)

This layer helps confirm:

- That suspicious activity is real
- That multiple indicators belong to the same incident so confirm the attack



Attack Scenarios Overview :

This section introduces the attack scenarios simulated in this project. Each scenario represents a real-world threat and attack types faced by organizations and is designed to demonstrate how a SOC can detect, analyze, and respond to different types of malicious activity using the different tools and centralized monitoring and log correlation.

The attacks were executed as scenarios, each with a distinct objective and detection focus. This approach allows the SOC to evaluate different threat categories without overlap or interception, ensuring clarity in analysis and reporting.

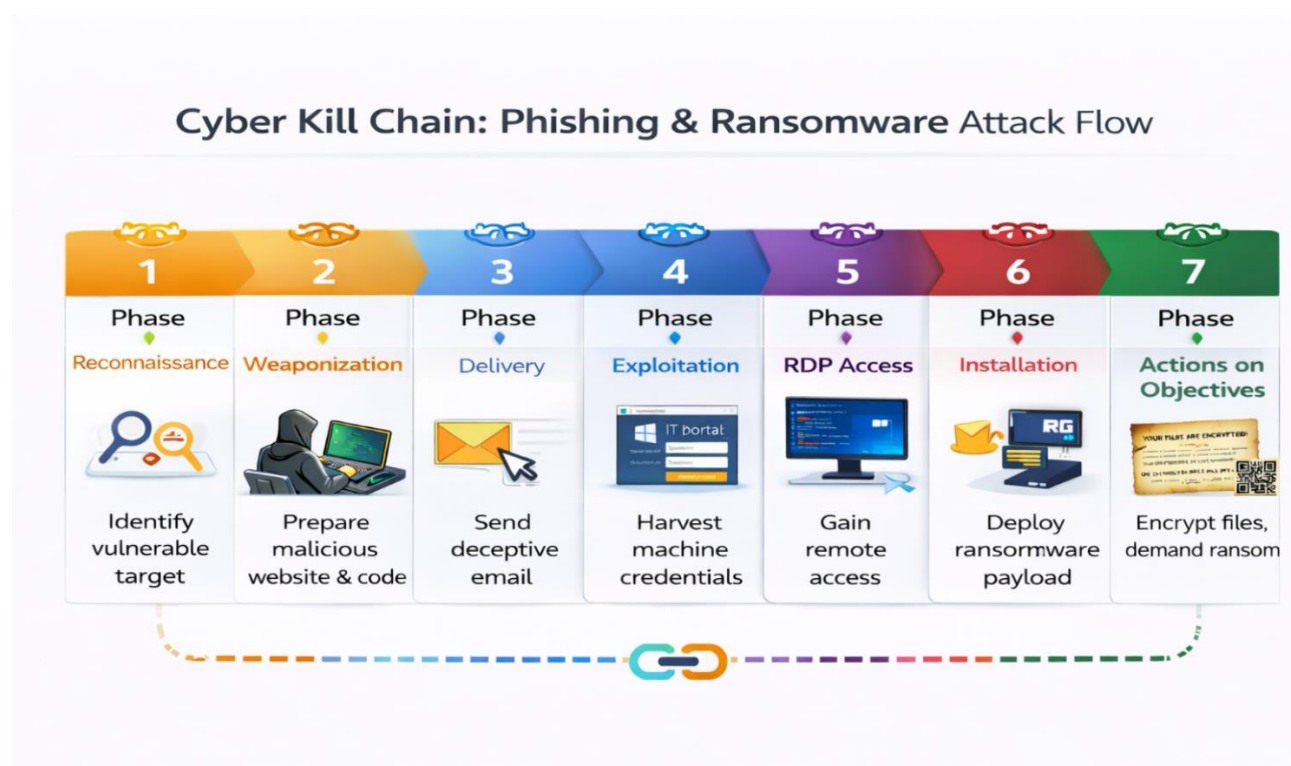
The following attack scenarios were simulated:

- **Phishing and Ransomware Attack**
- **Lateral Movement Attack (Honeypot Detection)**

Attack 1: Phishing and Ransomware Attack

Cyber Kill Chain Analysis & Execution

Overview: This simulation adopted the industry-standard Cyber Kill Chain framework to emulate a sophisticated, multi-stage attack. The objective was to compromise a specific target within the organization by impersonating the IT department, harvesting local machine credentials, and executing a ransomware payload via lateral movement.



Phase 1: Reconnaissance

Objective: Identification and profiling of a vulnerable target. Methodology: We shifted from broad passive gathering to continuous monitoring of the target's social media presence. By observing the employee's posting habits and online behavior over time, we identified a pattern of low security awareness and a tendency to click on external links without verification. This behavioral profile confirmed the employee as a high-probability target for a social engineering campaign.

Phase 2: Weaponization

Objective: Creation of tailored malicious payloads and infrastructure. Methodology: To guarantee access to the internal network, we designed a pretext that would

require the user to divulge their local machine credentials (Domain/Device Login) rather than standard web credentials.

1. Fake IT Portal (harvester.py): We developed a custom web page mimicking the organization's internal "IT Service Desk." The page displayed an urgent notification regarding a mandatory "Windows 11 Security Upgrade," explicitly requesting the user's *current* device username and password to authorize the remote installation.
2. Ransomware Simulation: A batch script the attacker will execute in the windows machine designed to simulate a cryptographic attack. Upon execution, the script will encrypt the sensitive file of the user and will not be able to be accessed until ransom are paid then the key will be given for decryption. triggers a system alert demanding payment.

```
(omar@omar)-[~]
$ nano it_update.html

(omar@omar)-[~]
$ nano it_update.html

(omar@omar)-[~]
$ nano harvester.py

(omar@omar)-[~]
$ python3 harvester.py

[*] Fake IT Service Desk running on port 8080
192.168.133.129 - - [25/Dec/2025 17:24:14] "GET / HTTP/1.1" 200 -
```

Figure 1: Development of the credential harvester server to stole the credentials.

Phase 3: Delivery

Objective: Transmission of the weaponized payload to the target via deceptive communication. Methodology: We employed a Typosquatting (or Homograph) Attack to deliver the payload. We registered a deceptive domain that visually mimicked the legitimate IT department's address (e.g., support@microsoft.com instead of microsoft.com).

- **The Email:** A spoofed email was sent to the victim with the subject line "ACTION REQUIRED: Device Update Scheduled."

- **The Lure:** The email contained a "legitimate-looking" link pointing to our malicious harvester, instructing the user to "Verify Credentials to Begin Update."

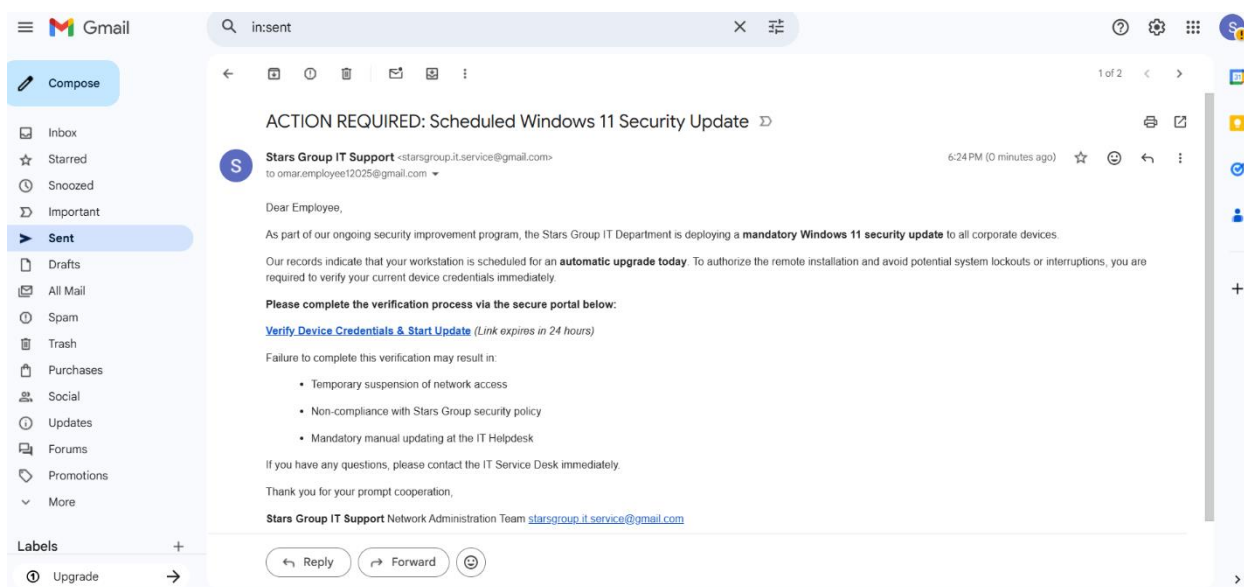


Figure 2: Delivery via Typosquatting: Impersonating IT support to distribute the malicious link.

Phase 4: Exploitation

Objective: Execution of the attack to compromise credentials. Methodology: Believing the request came from the IT department, the victim clicked the link and navigated to our fake portal. The harvester.py script successfully intercepted the HTTP POST request, capturing the Windows machine credentials (Username/Password) in cleartext on the attacker's console.

```
(omar@omar)~$ python3 harvester.py

[*] Fake IT Service Desk running on port 8080
192.168.133.129 - - [25/Dec/2025 17:24:14] "GET / HTTP/1.1" 200 -

[+] INTERCEPTED CREDENTIALS
    Username: omar
    Password: omar
192.168.133.129 - - [25/Dec/2025 17:24:41] "POST /login HTTP/1.1" 200 -
192.168.133.129 - - [25/Dec/2025 17:24:41] code 404, message Not Found
192.168.133.129 - - [25/Dec/2025 17:24:41] "GET /favicon.ico HTTP/1.1" 404 -
192.168.133.129 - - [25/Dec/2025 17:25:30] "GET / HTTP/1.1" 200 -
192.168.133.129 - - [25/Dec/2025 17:25:30] "GET / HTTP/1.1" 200 -
```

Figure 3: successful interception of local machine credentials via the fake IT portal.

Phase 5 Remote Desktop connection vis stolen credentials

Objective: Establishing persistent remote access. Methodology: With the valid machine credentials in hand, we bypassed the need for malware-based backdoors. We initiated a legitimate Remote Desktop Protocol (RDP) connection to the victim's machine. This simulated an attacker "living off the land"—using built-in administrative tools to pivot into the system undetected.

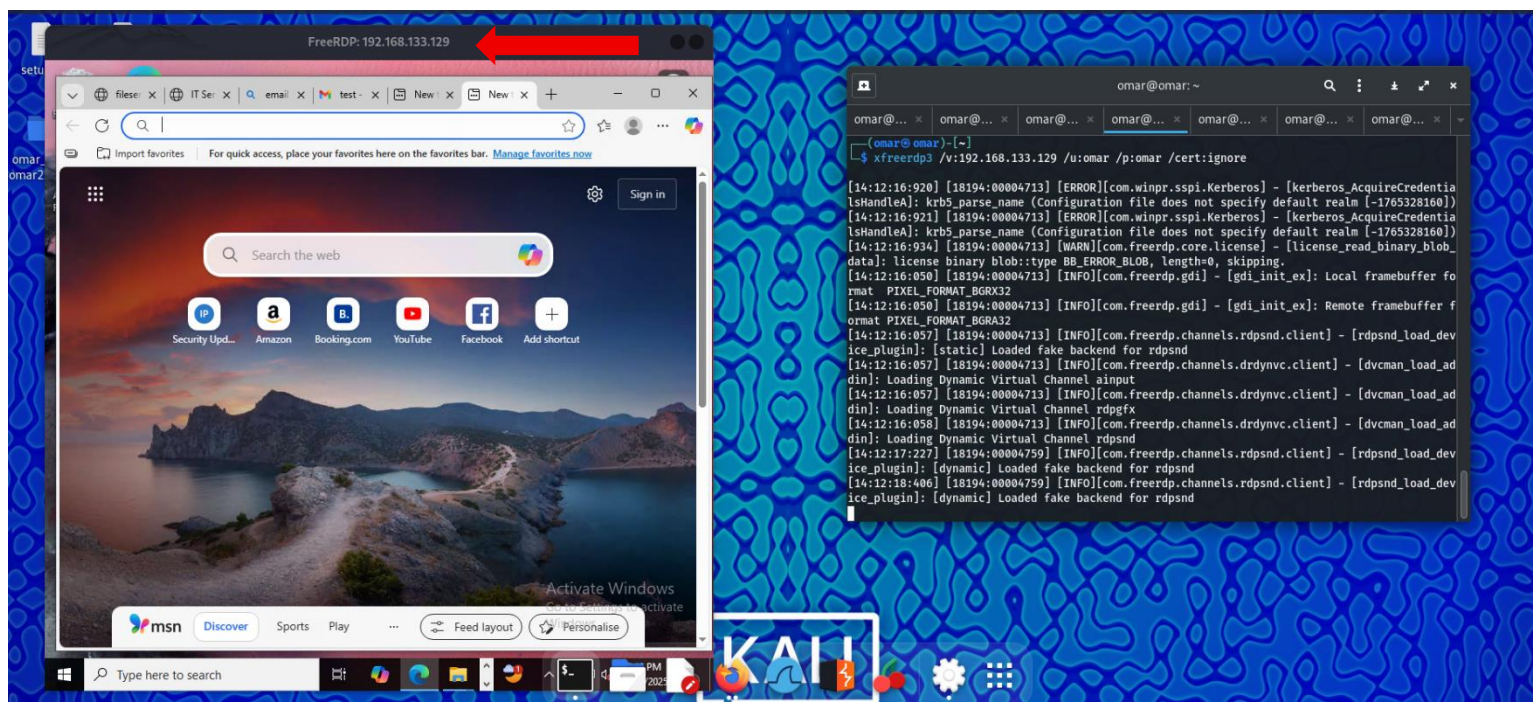


Figure 4: Unauthorized RDP session established using the harvested machine credentials.

Phase 6: Installation

Objective: Placing the ransomware payload on the victim machine. Methodology: We utilized Drive Redirection over RDP to transfer the ransom.bat payload. By mounting the attacker's file system as a network drive on the victim's machine, we blended our file transfer with legitimate RDP traffic. This method was chosen specifically to generate distinct Sysmon Event ID 11 logs where the source image is mstsc.exe, differentiating it from a standard web download.

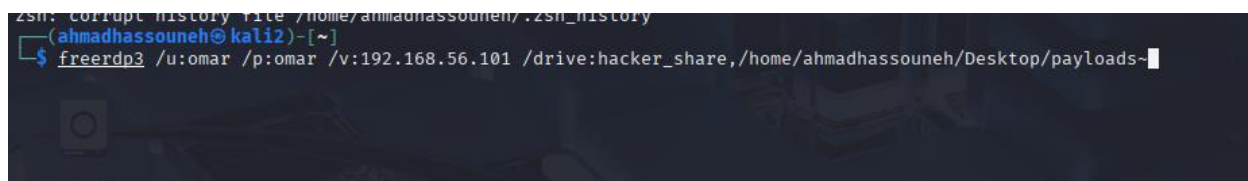


Figure 5: Lateral transfer of ransomware payload via RDP Drive Redirection.

Phase 7: Actions on Objectives

Objective: Execution of ransomware and simulated extortion. Methodology: With the payload installed, the attacker encrypts the passwords file. The script simulated the encryption of user data and launched a persistent pop-up message demanding a ransom payment to a specified Bitcoin wallet address. This action triggered high-severity alerts in our Splunk dashboard.

Before:



Figure 6:Text file before running the ransomware

After :

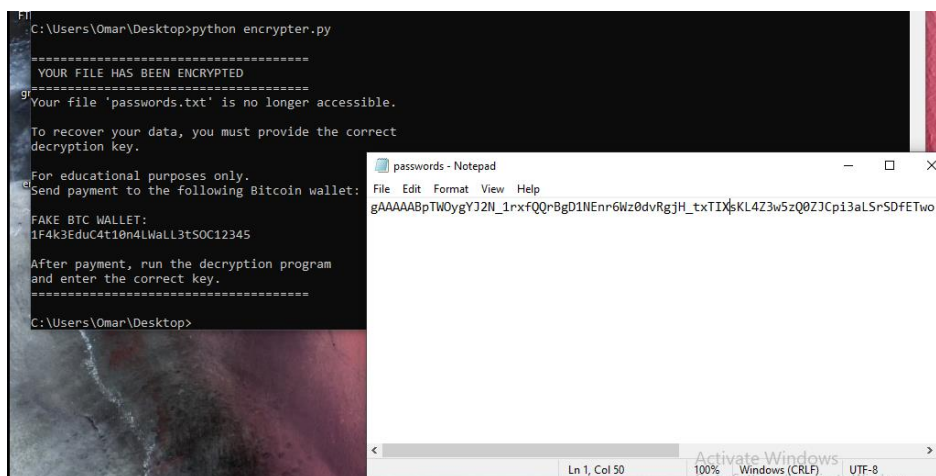


Figure 7: after running the ransomware

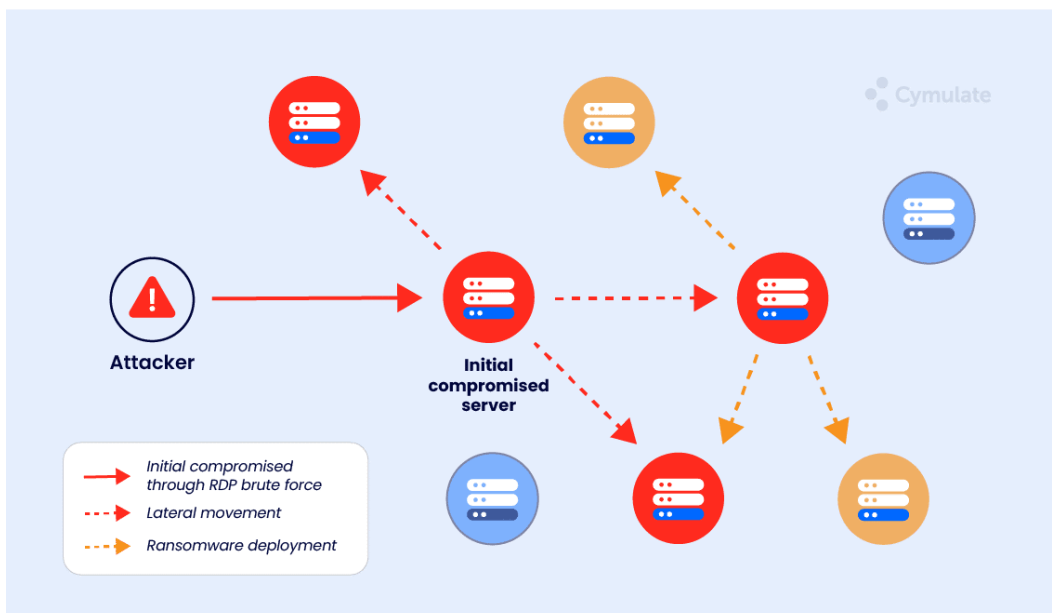
Phishing & Ransomware Attack Simulation



Attack 2 : Lateral Movement Attack (Honeytrap Detection)

1.Attack Overview

This attack scenario demonstrates how a Security Operations Center (SOC) can detect lateral movement attempts within an internal network after an endpoint has been compromised. Lateral movement is a common post-exploitation (what the attacker will do after gain access) technique where an attacker, after gaining access to one machine, attempts to discover and access other internal systems (compromise the windows machine then he will try to go further for sql server or fileserver). As shown in the figure below :



In this scenario, a Windows workstation is compromised by stolen credentials in order to get access via RDP port 3389 , so once the attacker got inside ; the attacker attempts to go further and compromise more machines and systems , then access what appears to be an internal file server. Unknown to the attacker, this internal service is actually a **honeypot (trap)** hosted on the SOC monitoring server.

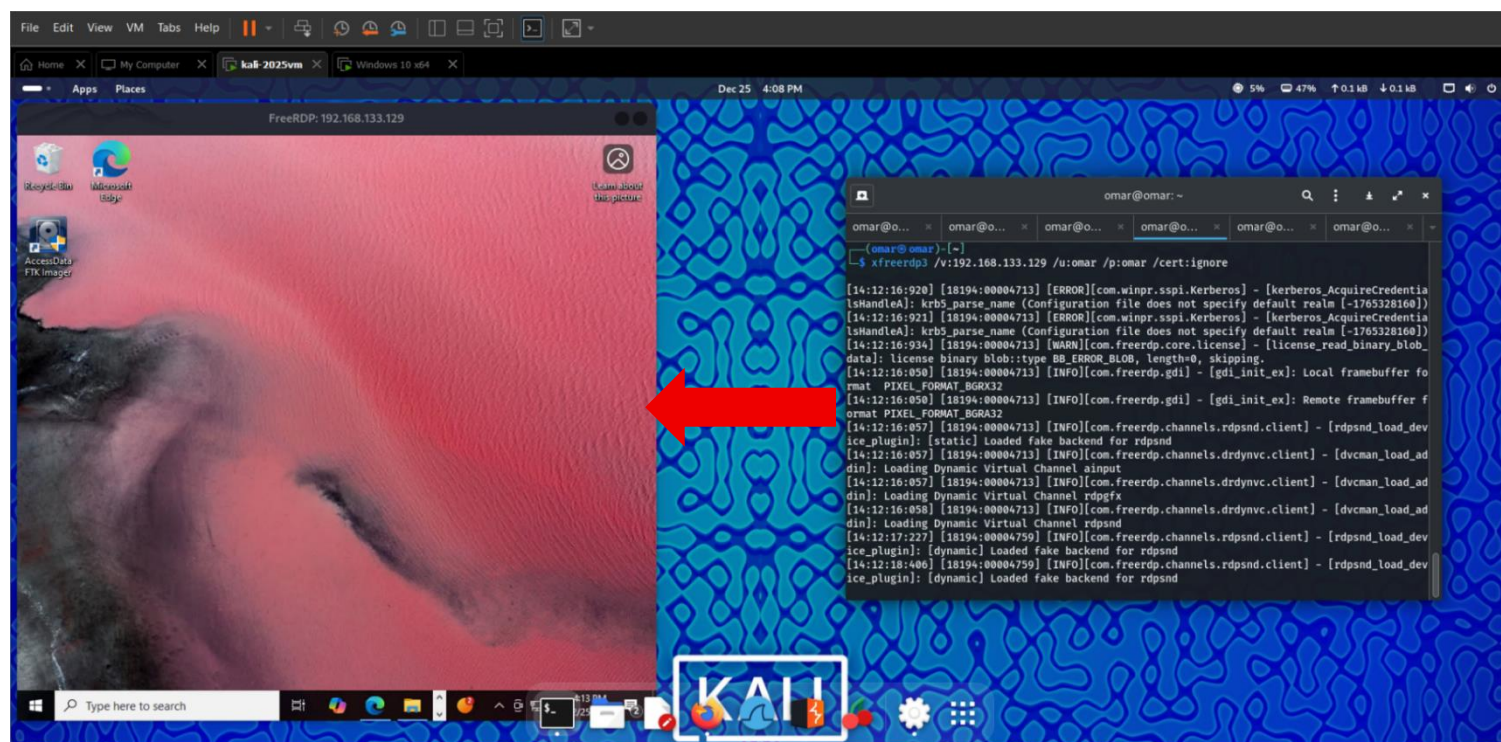
2. Initial Compromise and Attacker Access

The attacker gains remote access to the Windows endpoint using valid credentials, simulating a realistic compromise situation. Remote access is established using Remote Desktop Protocol (RDP), allowing the attacker to interact with the system as a legitimate user.

At this stage, the attacker has full control of the compromised endpoint and begins post-exploitation activities trying to go further and compromise more .

From the attacker's machine (Kali Linux), the following remote access command is used: **[xfreerdp3 /v:<Windows_IP> /u:<username> /p:<password> /cert:ignore]**

As shown on the ScreenShoot the attacker get access to the victim windows machine and its now accessible via kali (attacker machine)



3. Internal Discovery and Target Identification

Once inside the system, the attacker performs internal reconnaissance than scanning trying to find more machines to compromise since the attacker got access to internal system (the win machine) . To increase realism, a hostname (fileserver.local) is used instead of a direct IP address. This hostname is mapped to the honeypot system, simulating a legitimate internal file server. So now if the attacker try to access the fileserver to get sensitive files but the soc team was preparing a trap honeypot that mimics the fileserver name

Attacker → windows machine(RDP) →fileserver →honeypot

Screenshot shows the attacker trying to discover and finding the fileserver

```
PS C:\Windows\system32> ping fileserver.local

Pinging fileserver.local [192.168.133.128] with 32 bytes of data:
Reply from 192.168.133.128: bytes=32 time<1ms TTL=64
Reply from 192.168.133.128: bytes=32 time<1ms TTL=64
Reply from 192.168.133.128: bytes=32 time<1ms TTL=64
Reply from 192.168.133.128: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.133.128:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\Windows\system32> Test-NetConnection fileserver.local -Port 80

ComputerName      : fileserver.local
RemoteAddress     : 192.168.133.128
RemotePort        : 80
InterfaceAlias    : Ethernet1
SourceAddress     : 192.168.133.129
TcpTestSucceeded  : True
```

4. Honeypot Interaction

The honeypot, running on the monitoring server, is configured to listen on an internal service port and log all incoming connections made to the fileserver in order to trap the attacker . When the attacker attempts to access the internal server, the honeypot captures the request and records the source IP, timestamp, and connection details.

From the attacker's perspective, the service appears normal and giving the a files as samples " fake password.txt " , which is consistent with real internal systems protected by soc professionals to detect attack attempts .

Honeypot logs the unauthorized access attempt along with IPs and timestamps

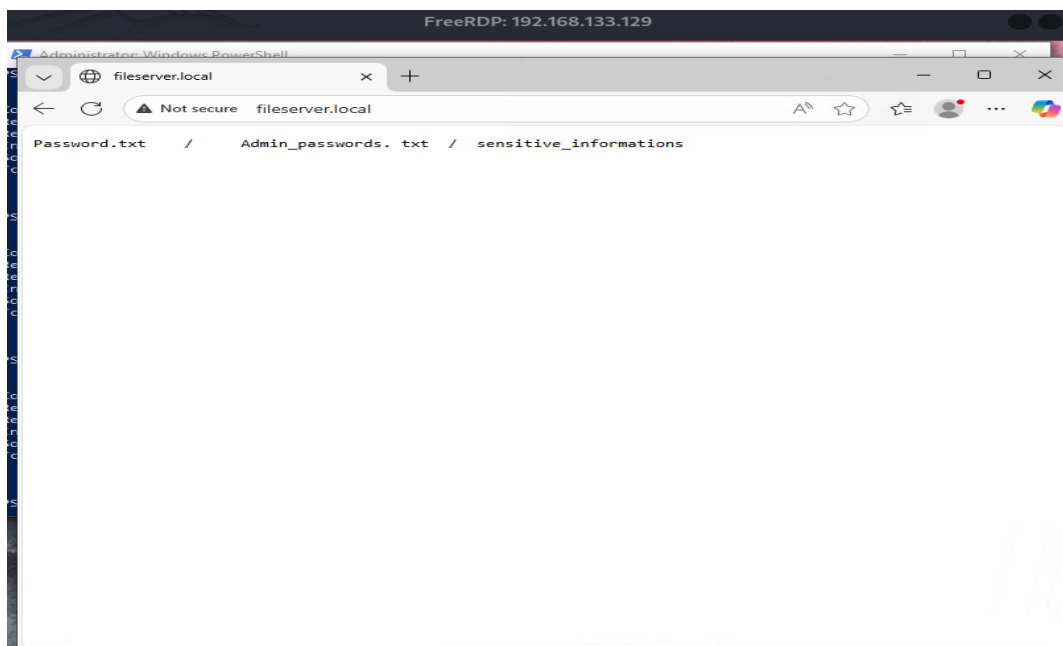
```
HONEYPOT ACTIVATED ON PORT 80 (2025-12-25 14:17:09 -0600)

INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49858 (2025-12-25 14:17:31 -0600)
-----

INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49934 (2025-12-25 14:36:06 -0600)
-----

INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49966 (2025-12-25 14:37:17 -0600)
-----
GET / HTTP/1.1
Host: fileserver.local
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apn
g,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
```

Fake files appear to the attacker



Logging and Detection Mechanisms

The lateral movement attempt generates multiple correlated logs:

- Sysmon (Windows endpoint) records the outbound network connection attempt (Event ID 3), including destination address and port.

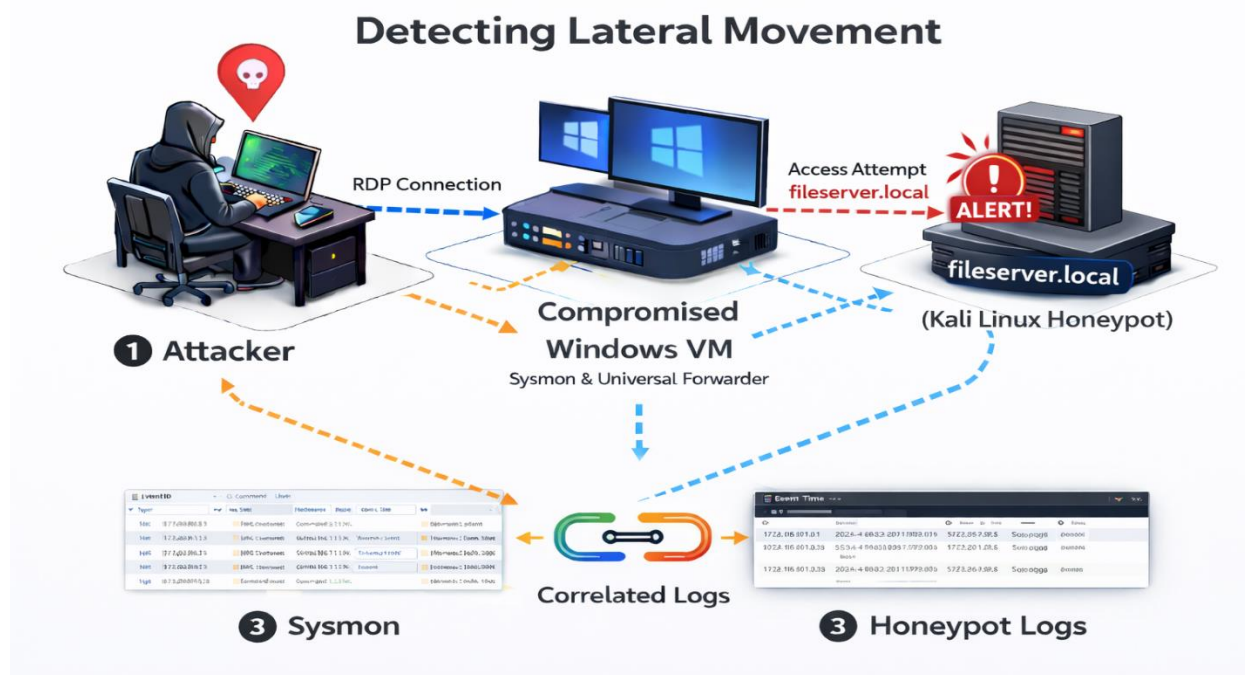
- Honeypot logs capture the inbound request and identify the compromised host as the source.
- Splunk SIEM ingests logs from both sources :
Sysmon / Honeypot
- allows the SOC analyst to correlate endpoint and network activity.

Honeypot logs sample

Format Show: 50 Per Page View: List

	Time	Event
		##### PentBox Honeypot log
6	12/25/25 2:17:09.000 PM	HONEYPOT ACTIVATED ON PORT 80 (2025-12-25 14:17:09 -0600)
7	12/25/25 2:17:31.000 PM	INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49858 (2025-12-25 14:17:31 -0600)
8	12/25/25 2:36:06.000 PM	INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49934 (2025-12-25 14:36:06 -0600)
9	12/25/25 2:37:17.000 PM	INTRUSION ATTEMPT DETECTED! from 192.168.133.129:49966 (2025-12-25 14:37:17 -0600)
		GET / HTTP/1.1 Host: fileserver.local

Lateral Movement scenario figure



SOC Perspective on the Attack Scenarios

From a SOC analyst's perspective, these scenarios represent **different threat categories** that show different attack tactics and techniques rather than a single attack chain . This separation allows analysts to:

- Investigate each incident independently (to find unique artifacts)
- Apply appropriate detection logic for each threat
- Reduce **false positives** by focusing on specific behaviors

Improve incident response **time** with minimum **resources**

Conclusion

This project presented the design and implementation of a Security Operations Center (SOC) lab within virtual environment using open-source and industry-relevant tools. The objective was to demonstrate how modern SOC's monitor, detect, and analyze security threats and many cyber attacks through centralized logging, endpoint visibility, network monitoring, and correlation.

A strategy was implemented using **Sysmon** for endpoint(win device) monitoring, **Splunk** as a SIEM platform, **Splunk Universal Forwarder** for log forwarding, and a **honeypot** for detecting internal threats by trapping the hacker. This architecture reflects real-world SOC practices and highlights the importance of visibility across multiple layers within the network , operating systems and endpoints of an organization's infrastructure.

Future Work

While the current implementation successfully demonstrates core SOC concepts, several enhancements and future work could be done in the future:

- Implementing automated alerts and correlation rules within Splunk
(using artificial intelligence) “ AI ”
- Expanding detection coverage to include additional threat behaviors such as data exfiltration or defense evasion
- Integrating **threat intelligence** feeds for enriched analysis
- Simulating multiple endpoints to reflect more complex environments
- Enhancing dashboards for real-time SOC monitoring and executive reporting



THE END

